

# Setting up the raspberry pi

27 April 2026 12:32

-> Need a boot media which contains the OS image, here the boot media preferred is micro SD card, with min 32GB storage.

-> Power Cable to be used is USB to type C.

-> Before powering on your Raspberry Pi you must flash (write) the operating system (OS) image onto the storage for your device, otherwise your Raspberry Pi won't start.

## Things to check before flashing and connecting any board

- **Power supply**  
Make sure voltage/current meet the board's requirement. Unstable power causes random failures.
- **Boot medium**  
Know where the board boots from (SD card, eMMC, USB, flash). Prepare that medium correctly.
- **Flashing tool & image**  
Use the correct tool and OS/firmware for that specific board (wrong image = no boot).
- **Interface you'll use**  
Decide beforehand:
  - Headless (SSH)
  - HDMI/monitor
  - Serial (UART)Always keep **UART as backup** if possible.
- **Network plan (very important)**
  - Prefer simple Wi-Fi (2.4 GHz or hotspot) for first boot
  - Ensure correct SSID/password
  - Make sure your PC and board will be on the same network
- **Enable access method**  
If headless:
  - Enable SSH during flashing
  - Set username/password
- **Driver/port readiness on PC**
  - USB drivers installed
  - Serial ports detectable
  - Storage devices writable
- **Basic verification tools ready**
  - Network scanner
  - Ping
  - SSH client
- **Understand expected indicators**
  - LEDs (boot status)
  - Serial logs (if available)

## Simple takeaway

Before flashing any board, always plan:

Power → Boot medium → Flashing → Access method → Network

## Post-setup checks and why they matter

- **hostname, whoami**  
Confirm you are logged into the correct board, not your PC.
- **ip a, ping google.com**  
Verify the board has a valid IP and internet access.
- **sudo apt update, sudo apt upgrade**

Ensure system packages are up to date and stable.

- **uname -a, df -h, free -h**

Check OS version, storage space, and memory status.

- **Basic file test (touch, ls)**

Confirm filesystem is working correctly.

- **Install tools (build-essential, git) :** sudo apt install build-essential git -y

Prepare the system for development work.

- **raspi-config (enable interfaces):** sudo raspi-config

Enable required hardware features like GPIO, I2C, SPI.

- **Basic hardware test**

Ensure peripherals and board interfaces are functioning.

### Simple takeaway

Each step was to confirm:

Correct system → network → stability → resources → readiness for development

How to login to ssh through cmd prompt:

ssh username@raspberrypi.local

or

ssh username@<IP\_ADDR>

At Present login deets:

ssh digicomm@digicomm.local

pwd: Digicomm@2025.

# errors

28 April 2026 17:40

## SD Card Debugging :

### 1. SD card not detected properly on my PC

- Tried on another PC → detected
- Conclusion: issue is in PC, not the card

### 2. Not sure if SD card is usable

- Created file → removed → reinserted → file still there
- Conclusion: read/write working, SD card is healthy

### 3. Drive not visible in File Explorer

- Seen only in Disk Management
- Assigned drive letter
- Result: drive became visible

### 4. "Error writing to storage device"

- Checked physical lock → OK
- Checked diskpart → not read-only
- Conclusion: not a lock/read-only issue

### 5. Suspected SD card damage

- Re-tested on another PC (format + write + read)
- Conclusion: SD card confirmed good

### 6. Write operations behaving abnormally

- Checked using:

diskpart

list disk

select disk

attributes disk

- Found: disk not read-only

### 7. clean command hanging

- Changed USB port
- Replugged device
- Result: clean worked instantly
- Conclusion: USB port/connection issue

### 8. Resetting SD card

- Executed:

clean

create partition primary

format fs=fat32 quick

assign

- Result: SD card fully reset and usable

### 9. Unable to eject ("device in use")

- Closed apps, restarted explorer, waited
- Conclusion: Windows holding background process
- Action: waited and removed safely

### 10. Flashing OS

- Used Raspberry Pi Imager
- Confusion: advanced settings shortcut didn't open
- Realization: settings were already shown during write
- Result: OS successfully written

## Final Outcome

- SD card → working
- USB/port issue → fixed
- Disk → cleaned and reset
- OS flashing → successful

Issue was with PC's USB handling, not the SD card. Changing port and cleaning the disk resolved everything.

//Connect to phone wifi

How to login to ssh through cmd prompt:

ssh username@raspberrypi.local

Hostname: raspberrypi

username: chethanam

pwd : cheth252

SSID: S25

#### **Wi-Fi Connection Issue and Solution**

- The Raspberry Pi failed to connect to the initial network (Digicomm\_ODC\_5G) during boot.
- Likely reasons: 5 GHz network compatibility issues or incorrect Wi-Fi credentials.
- Because of this, the Pi did not get an IP address and was not visible on the network.
- Solution: switched to a mobile hotspot with simple SSID and password.
- Result: Pi connected successfully, obtained an IP address, and SSH access worked.

# Lib / functions info

05 May 2026 12:48

To write GPIO-based programs in modern Linux systems, the libgpiod v2 library is used. It provides controlled access to GPIO pins through a request-based model.

## GPIO Core:

In libgpiod v2, the GPIO system is organized as follows:

- Chip represents the entire GPIO controller
- Line represents a GPIO pin conceptually, but it is not directly accessed in v2

### gpiod\_chip\_open()

```
struct gpiod_chip *gpiod_chip_open(const char *path);
```

- path → device path of GPIO controller (example: "/dev/gpiochip0")
- Returns a pointer to the GPIO chip structure

This function opens access to the GPIO controller and is required before any GPIO operations.

**Line Settings:** Line settings define how a GPIO pin behaves.

### gpiod\_line\_settings\_set\_direction()

```
void gpiod_line_settings_set_direction(struct gpiod_line_settings *settings, enum gpiod_line_direction direction);
```

- settings → pointer to line settings object
- direction → input or output direction

Used to define whether a GPIO pin acts as input or output.

**Line Configuration:** GPIO pins are grouped and configured using a line configuration object.

### gpiod\_line\_config\_add\_line\_settings()

```
void gpiod_line_config_add_line_settings(  
    struct gpiod_line_config *config,  
    const unsigned int *offsets,  
    size_t num_offsets,  
    struct gpiod_line_settings *settings  
);
```

- config → line configuration object
- offsets → array of GPIO pin numbers
- num\_offsets → number of GPIO pins in the array
- settings → configuration applied to those pins

This function assigns settings (input/output) to specific GPIO pins.

**Request System:** GPIO control is performed through a request object.

### gpiod\_chip\_request\_lines()

```
struct gpiod_line_request *gpiod_chip_request_lines(  
    struct gpiod_chip *chip,  
    struct gpiod_request_config *req_cfg,  
    struct gpiod_line_config *line_cfg  
);
```

- chip → GPIO controller
- req\_cfg → request configuration (consumer name, etc.)
- line\_cfg → GPIO line configuration

Returns a request object used for all GPIO operations.

### gpiod\_request\_config\_set\_consumer()

```
void gpiod_request_config_set_consumer(  
    struct gpiod_request_config *config,  
    const char *consumer  
);
```

- config → request configuration object
- consumer → name of the application using GPIO

## GPIO Output/Input Operations

### gpiod\_line\_request\_set\_value()

```
int gpiod_line_request_set_value(
    struct gpiod_line_request *request,
    unsigned int offset,
    int value
);
```

- request → active GPIO request object
- offset → GPIO pin number
- value → 1 (HIGH) or 0 (LOW)

Used to set GPIO output state.

```
gpiod_line_request_get_value()
int gpiod_line_request_get_value(
    struct gpiod_line_request *request,
    unsigned int offset
);
```

- request → active GPIO request object
- offset → GPIO pin number

Returns:

- 1 → HIGH
- 0 → LOW

Used to read GPIO input state.

## Resource Management

```
gpiod_line_request_release()
void gpiod_line_request_release(
    struct gpiod_line_request *request
);
```

- request → active GPIO request object

Releases GPIO resources.

```
gpiod_chip_close()
void gpiod_chip_close(struct gpiod_chip *chip);
```

- chip → GPIO controller object

Closes access to GPIO hardware.

## Timer Core

```
clock_gettime()
int clock_gettime(clockid_t clk_id, struct timespec *tp);
```

- clk\_id → clock source (usually CLOCK\_MONOTONIC)
- tp → pointer to timespec structure storing time

Provides high precision time in nanoseconds.

```
usleep()
int usleep(useconds_t usec);
```

- usec → time to sleep in microseconds

Pauses program execution temporarily.